

Karate Club: Um Arcabouço Python de Código Aberto Orientado a API para Aprendizado Não Supervisionado em Grafos

Benedek Rozemberczki
Universidade de Edimburgo
Edimburgo, Reino Unido
benedek.rozemberczki@ed.ac.uk

Oliver Kiss
Universidade Central Europeia
Budapeste, Hungria
kiss_oliver@phd.ceu.edu

Rik Sarkar
Universidade de Edimburgo
Edimburgo, Reino Unido
rsarkar@inf.ed.ac.uk

RESUMO

Os grafos codificam propriedades estruturais importantes de sistemas complexos. O aprendizado de máquina em grafos surgiu, portanto, como uma técnica importante em pesquisa e aplicações. Apresentamos o Karate Club – um arcabouço (framework) em Python que combina mais de 30 algoritmos de mineração de grafos no estado da arte. Essas técnicas não supervisionadas facilitam a identificação e representação de características comuns de grafos. O objetivo primário do pacote é tornar a detecção de comunidades, a incorporação (embedding) de nós e de grafos inteiros disponíveis para um público amplo de pesquisadores e profissionais de aprendizado de máquina. O Karate Club é projetado com ênfase em uma interface de aplicação consistente, escalabilidade, facilidade de uso, comportamento padrão robusto dos modelos, ingestão padronizada de conjuntos de dados e geração de saídas. Este artigo discute os princípios de design por trás do arcabouço com exemplos práticos. Demonstramos a eficiência do Karate Club no desempenho de aprendizado em uma ampla gama de problemas reais de agrupamento (clustering) e tarefas de classificação, juntamente com evidências de suporte sobre sua velocidade competitiva.

KEYWORDS

incorporação de redes, incorporação de grafos, aprendizado de representação, análise de redes, mineração de grafos

ACM Reference Format:

Benedek Rozemberczki, Oliver Kiss, and Rik Sarkar. 2020. Karate Club: Um Arcabouço Python de Código Aberto Orientado a API para Aprendizado Não Supervisionado em Grafos. In *Proceedings of the 29th ACM International Conference on Information and Knowledge Management (CIKM '20)*, October 19–23, 2020, Virtual Event, Ireland. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3340531.3412757>

1 INTRODUÇÃO

Técnicas que extraem características (features) de dados em grafos de maneira não supervisionada [19, 23, 44] têm alcançado um sucesso crescente na comunidade de aprendizado de máquina. As

características extraídas automaticamente por esses métodos podem servir como entradas para predição de links, classificação de nós e de grafos, detecção de comunidades e várias outras tarefas [8, 23, 24, 29] em uma ampla gama de cenários de pesquisa e aplicação do mundo real. Ferramentas de mineração de grafos como [13, 17, 22] contribuíram para a melhoria e desenvolvimento de pipelines de aprendizado de máquina. A necessidade de engenharia de características personalizada e complexa é reduzida por técnicas não supervisionadas de mineração de grafos. Essa abordagem produz características que são naturalmente reutilizáveis em múltiplos tipos de tarefas. Pesquisas recentes [23, 24, 35] tornaram essa extração de características altamente eficiente e paralelizável.

A democratização do aprendizado de máquina para dados tabulares foi liderada por frameworks que viabilizaram o desenvolvimento rápido. Ferramentas como [1, 5, 21] estão disponíveis em linguagens de script de propósito geral com interfaces consistentes e fáceis de usar. Por outro lado, os frameworks de aprendizado baseados em grafos atuais são menos maduros e de escopo limitado [13, 22]. Por exemplo, esses pacotes hospedam certos algoritmos de detecção de comunidades, mas nenhum para incorporação (embedding) de nós ou de grafos inteiros. Além disso, algumas dessas ferramentas [17, 22] impõem barreiras significativas para os usuários finais em termos de instalação de pré-requisitos e estruturas de dados personalizadas usadas para representar grafos.

Trabalho presente. Propomos o Karate Club, um framework Python de código aberto para aprendizado não supervisionado em grafos. Implementamos o Karate Club com princípios de design consistentes e orientados a API em mente, o que torna a biblioteca amigável ao usuário final e modular. O nome do pacote é inspirado no Zachary's Karate Club [45] – uma rede comumente usada para demonstrar algoritmos de redes. O design desta caixa de ferramentas de aprendizado de máquina foi motivado pelos princípios usados para criar o amplamente utilizado pacote scikit-learn [5].

O design envolve uma série de padrões fundamentais de engenharia. Cada algoritmo possui uma configuração padrão de hiperparâmetros sensível que ajuda profissionais não especialistas. Para facilitar ainda mais o uso do nosso pacote, os algoritmos têm um número limitado de métodos públicos compartilhados (por exemplo, fit). Os modelos ingerem estruturas de dados do ecossistema científico do Python [13, 37, 38] como entrada e a saída gerada também segue esses formatos. A mecânica interna do modelo é sempre implementada como métodos privados usando bibliotecas de back-end otimizadas [9, 28, 37, 38] para computação. Esses princípios, combinados com a documentação extensiva, garantem que o Karate Club seja acessível a um público mais amplo do que os frameworks de mineração de grafos de código aberto atualmente disponíveis.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

CIKM '20, October 19–23, 2020, Virtual Event, Ireland

© 2020 Association for Computing Machinery.

ACM ISBN 978-1-4503-6859-9/20/10...\$15.00

<https://doi.org/10.1145/3340531.3412757>

Nossa avaliação empírica se concentra em três tarefas comuns de mineração de grafos: detecção de comunidades disjuntas (sem sobreposição), classificação de nós e de grafos. Comparamos o desempenho de aprendizado de algoritmos em nível de nó e de grafo implementados em nosso framework em várias redes sociais, da web e de colaboração do mundo real (coletadas do Deezer, Reddit, Facebook, Twitch, Wikipedia e GitHub). Em relação ao tempo de execução, os modelos no Karate Club mostram uma escalabilidade razoável, que demonstramos pelo uso de dados sintéticos.

Nossas contribuições. Especificamente, nosso trabalho traz as seguintes contribuições principais:

- (1) Lançamos o Karate Club, um framework Python de mineração de grafos que fornece uma ampla gama de procedimentos fáceis de usar para detecção de comunidades, incorporação de nós e de grafos inteiros.
- (2) Demonstramos com código as principais ideias do design do framework orientado a API: encapsulamento e inspeção de hiperparâmetros, métodos públicos disponíveis, ingestão de dados, geração de saídas e interface com algoritmos de aprendizado downstream e métodos de avaliação.
- (3) Avaliamos o desempenho de aprendizado do framework em problemas reais de detecção de comunidades, classificação de nós e classificação de grafos. Validamos a escalabilidade dos algoritmos implementados no framework.
- (4) Disponibilizamos em código aberto, junto com o framework, uma documentação detalhada e múltiplos conjuntos de dados de classificação de grafos de grande porte.

O restante deste artigo está estruturado da seguinte forma. Na Seção 2, discutimos as técnicas de mineração de grafos cobertas. Apresentamos uma visão geral dos principais princípios por trás do Karate Club na Seção 3, onde incluímos exemplos detalhados para explicar essas ideias de design. O desempenho de aprendizado e a escalabilidade dos algoritmos no pacote são avaliados na Seção 4. Revisamos bibliotecas de mineração de dados relacionadas na Seção 5. O artigo conclui com a Seção 6, onde discutimos direções futuras.

2 PROCEDIMENTOS DE MINERAÇÃO DE GRAFOS NO KARATE CLUB

Nesta seção, discutimos brevemente os vários algoritmos de mineração de grafos que estão disponíveis no lançamento 1.0 do pacote Karate Club.

2.1 Detecção de Comunidades

As técnicas de detecção de comunidades agrupam os vértices do grafo em conjuntos de nós densamente conectados. Esse agrupamento pode ser o resultado final ou uma entrada para um algoritmo de aprendizado downstream (por exemplo, classificação de nós ou predição de links).

O Karate Club atualmente contém vários métodos para detecção de comunidades com e sem sobreposição. A detecção de comunidades sem sobreposição é análoga ao agrupamento clássico (clustering) e assume que um nó só pode pertencer a um único grupo; ver, por exemplo, [18, 25, 27, 30]. Enquanto a detecção de comunidades com sobreposição é análoga ao agrupamento difuso (fuzzy

clustering), pois os nós possuem afiliação com múltiplos clusters; ver, por exemplo, [16, 33, 39, 44].

2.2 Incorporação de Nós (Node Embedding)

As incorporações de nós mapeiam os vértices de um grafo em um espaço euclidiano no qual aqueles que são considerados semelhantes de acordo com uma certa noção estarão em estreita proximidade. A representação euclidiana facilita a aplicação de bibliotecas padrão de aprendizado de máquina para classificação de nós, predição de links, agrupamento, etc.

Incorporação que preserva a vizinhança mantém a proximidade dos nós no grafo quando um embedding é criado. Esses métodos decompõem implicitamente [23, 24, 31] ou explicitamente [6, 15, 26, 34] uma matriz de proximidade para criar a incorporação do nó.

Incorporação estrutural conserva os papéis estruturais dos nós no espaço de incorporação [2, 10, 14]. Nós com embeddings semelhantes têm uma distribuição semelhante de estatísticas descritivas (por exemplo, centralidade e coeficiente de agrupamento) em sua vizinhança.

Incorporação atribuída retém a estrutura da vizinhança e a similaridade de características genéricas dos nós quando a incorporação é aprendida. Esse aprendizado envolve a fatoração conjunta das matrizes nó-nó e nó-característica com uma técnica de decomposição de matriz direta [40, 43] ou implícita [29, 46].

Meta-incorporação combina informações de embeddings atribuídos, estruturais e que preservam a vizinhança para criar embeddings com maior qualidade de representação [41].

2.3 Incorporação e Sumarização de Grafos Inteiros

As técnicas de incorporação e sumarização de grafos inteiros criam representações de tamanho fixo de grafos inteiros como pontos em um espaço euclidiano. Os grafos que estão próximos no espaço de incorporação compartilham padrões estruturais, como subárvores.

Assinaturas espectrais (spectral fingerprints) extraem estatísticas dos autovetores e autovalores do Laplaciano do grafo [8, 35, 36]. Vetores das estatísticas descritivas são usados como a representação do grafo inteiro.

Técnicas de fatoração implícita criam uma matriz grafo-característica estrutural [7, 19] enumerando características textuais nos grafos. Esta matriz é decomposta para criar descritores de grafos inteiros e embeddings de características de forma conjunta.

3 PRINCÍPIOS DE DESIGN

Quando criamos o Karate Club, usamos um ponto de vista de design de sistemas de aprendizado de máquina orientado a API [5] para tornar a ferramenta amigável ao usuário final. Este princípio de design orientado a API envolve algumas ideias simples. Nesta seção, discutimos essas ideias e suas vantagens aparentes com exemplos ilustrativos apropriados em grande detalhe.

3.1 Hiperparâmetros do Modelo Encapsulados e Inspeção

Uma instância não supervisionada do modelo do Karate Club é criada usando o construtor do objeto Python apropriado. Este construtor possui uma configuração padrão de hiperparâmetros que permite o uso imediato e sensível do modelo. Definimos esses hiperparâmetros padrão para fornecer um aprendizado e um tempo de execução razoáveis. Se necessário, esses hiperparâmetros podem ser modificados no momento da criação da instância do modelo com a parametrização apropriada do construtor. Os hiperparâmetros são armazenados como atributos públicos para permitir a inspeção das configurações do modelo.

Demonstramos o encapsulamento de hiperparâmetros pelo trecho de código na Figura 1. Primeiro, queremos criar uma incorporação para um grafo Erdos-Renyi gerado pelo NetworkX com as configurações padrão. Quando o modelo é construído e ajustado, não alteramos os hiperparâmetros padrão e podemos imprimir o valor padrão do hiperparâmetro de dimensões. Segundo, decidimos definir um número diferente de dimensões, então criamos e ajustamos um novo modelo e imprimimos o novo valor do hiperparâmetro de dimensões.

```
1 import networkx as nx
2 from karateclub import DeepWalk
3
4 graph = nx.gnm_random_graph(100, 1000)
5
6 # Modelo padrão
7 model = DeepWalk()
8 model.fit(graph)
9 print(model.dimensions) # Saída padrão: 128
10
11 # Modelo customizado
12 model = DeepWalk(dimensions=64)
13 model.fit(graph)
14 print(model.dimensions) # Saída: 64
```

Figura 1: Criação de um grafo sintético e uso do modelo DeepWalk com hiperparâmetros padrão e modificados.

3.2 Consistência da API e Não Proliferação de Classes

Cada modelo de aprendizado de máquina não supervisionado no Karate Club é implementado como uma classe separada que herda da classe Estimator. Os algoritmos implementados em nosso framework possuem um número limitado de métodos públicos. Todos os modelos são treinados pelo uso do método `fit`, que recebe as entradas (grafo, características dos nós) e aprende o modelo. Os embeddings de nós e de grafos são retornados pelo método público `get_embedding`, e as afiliações de comunidades são recuperadas chamando `get_memberships`.

Evitamos a proliferação de classes com duas estratégias específicas. Primeiro, as entradas usadas por nosso framework e as saídas geradas não dependem de classes de dados personalizadas. Segundo, algoritmos que usam o mesmo pré-processamento de dados

ou etapa algorítmica foram construídos sobre blocos compartilhados.

Na Figura 2, criamos um grafo aleatório, um modelo DeepWalk com os hiperparâmetros padrão, ajustamos este modelo usando o método público `fit` e retornamos a incorporação chamando o método público `get_embedding`.

```
1 import networkx as nx
2 from karateclub import DeepWalk
3
4 graph = nx.gnm_random_graph(100, 1000)
5
6 model = DeepWalk()
7 model.fit(graph)
8 embedding = model.get_embedding()
```

Figura 2: Criação de um grafo sintético, ajuste do modelo DeepWalk e retorno do embedding.

O exemplo na Figura 2 pode ser modificado para criar uma incorporação Walklets com esforço mínimo, alterando a importação do modelo e o construtor, como mostrado na Figura 3.

```
1 import networkx as nx
2 from karateclub import Walklets
3
4 graph = nx.gnm_random_graph(100, 1000)
5
6 model = Walklets()
7 model.fit(graph)
8 embedding = model.get_embedding()
```

Figura 3: Criação de um grafo sintético, ajuste do modelo Walklets e retorno do embedding.

3.3 Ingestão Padronizada de Conjuntos de Dados

Projetamos o Karate Club para usar a ingestão padronizada de conjuntos de dados quando um modelo é ajustado. Praticamente, isso significa que algoritmos que possuem o mesmo propósito usam os mesmos tipos de dados para o treinamento do modelo:

- Técnicas de incorporação de nós estruturais e baseadas em vizinhança usam um único grafo NetworkX como entrada para o método `fit`.
- Procedimentos de incorporação de nós atribuídos recebem um grafo NetworkX como entrada e as características são representadas como um array NumPy ou como uma matriz esparsa SciPy.
- Métodos de incorporação no nível do grafo e assinaturas estatísticas de grafos recebem uma lista de grafos NetworkX como entrada.
- Métodos de detecção de comunidades usam um grafo NetworkX como entrada.

3.4 Mecânica do Modelo de Alto Desempenho

A mecânica subjacente dos algoritmos de mineração de grafos foi implementada usando bibliotecas Python amplamente disponíveis que não dependem do sistema operacional. As representações internas de grafos no Karate Club usam o NetworkX. Operações de álgebra linear densa são feitas com o NumPy e suas contrapartes esparsas usam o SciPy. Técnicas de fatoração de matriz implícita utilizam o pacote GenSim e os métodos que dependem de processamento de sinais em grafos usam o PyGSP.

3.5 Geração de Saída Padronizada e Integração Downstream

A geração de saída padronizada do Karate Club garante que os algoritmos de aprendizado não supervisionado que servem ao mesmo propósito sempre retornem o mesmo tipo de saída com uma ordenação consistente dos pontos de dados:

- Algoritmos de incorporação de nós sempre retornam um array float do NumPy quando o método `get_embedding` é chamado.
- Métodos de incorporação de grafos inteiros retornam um array float do NumPy quando o método `get_embedding` é chamado.
- Procedimentos de detecção de comunidades retornam um dicionário quando o método `get_memberships` é chamado.

Demonstramos a geração de saída padronizada e a integração pelo trecho de código na Figura 4. Criamos agrupamentos de um grafo aleatório e retornamos dicionários contendo as afiliações de cluster. Usando a biblioteca externa `community`, podemos calcular a modularidade desses agrupamentos.

```

1 import community
2 import networkx as nx
3 from karateclub import LabelPropagation, SCD
4
5 graph = nx.gnm_random_graph(100, 1000)
6
7 model = SCD()
8 model.fit(graph)
9 scd_memberships = model.get_memberships()
10
11 model = LabelPropagation()
12 model.fit(graph)
13 lp_memberships = model.get_memberships()
14
15 print(community.modularity(scd_memberships, graph))
16 print(community.modularity(lp_memberships, graph))

```

Figura 4: Agrupamento com duas técnicas de detecção de comunidades e uso de uma biblioteca externa para avaliar a modularidade.

3.6 Limitações

O design atual do Karate Club apresenta certas limitações e assume premissas fortes sobre a entrada. Assumimos que o grafo do

NetworkX é não direcionado e consiste em um único componente fortemente conexo. Todos os algoritmos assumem que os nós são indexados com inteiros consecutivamente e o índice do nó inicial é 0. Além disso, assumimos que o grafo não é multipartido, os nós são homogêneos e as arestas não têm peso.

4 AVALIAÇÃO EXPERIMENTAL

Na avaliação experimental do Karate Club, demonstraremos a qualidade dos embeddings e das comunidades extraídas em problemas reais e validaremos a escalabilidade linear na prática.

4.1 Desempenho de Aprendizado

A avaliação da qualidade da representação foca em três tarefas: detecção de comunidades com comunidades de referência (ground truth), classificação de nós e classificação de grafos inteiros.

4.1.1 Conjuntos de Dados. Para avaliar o desempenho dos algoritmos no nível do nó, usamos redes atribuídas que estão publicamente disponíveis no SNAP [17]. Esses conjuntos de dados são resumidos na Tabela 1.

Tabela 1: Redes sociais usadas para algoritmos no nível do nó.

Métrica	Wikipedia	GitHub	Twitch	FB Page-Page
Nós	11.631	37.700	7.126	22.470
Densidade	0,003	0,001	0,002	0,001
Transitividade	0,026	0,013	0,042	0,232
Diâmetro	11	7	10	15
Atributos	13.183	4.005	2.545	4.714

Os conjuntos de dados para nível de grafo estão descritos na Tabela 2.

Tabela 2: Estatísticas dos conjuntos de dados de grafos.

Dataset	Grafos	Nós (Min/Max)	Densidade	Diâmetro
Reddit	203.088	11 / 97	0,02 / 0,38	2 / 27
Twitch	127.094	14 / 52	0,03 / 0,96	1 / 2
GitHub	12.725	10 / 957	0,00 / 0,56	2 / 18
Deezer	9.629	11 / 363	0,01 / 0,90	2 / 2

4.1.2 Detecção de Comunidades. Relatamos na Tabela 3 as médias de NMI com os erros padrão calculados a partir de 100 execuções experimentais.

4.1.3 Classificação de Grafos. Os resultados médios de AUC são apresentados na Tabela 4.

4.1.4 Classificação de Nós. As médias de AUC com erros padrão são reportadas na Tabela 5.

Tabela 3: Valores médios de NMI com erros padrão (100 runs).

Algoritmo	Wikipedia	GitHub	Twitch	FB Page-Page
DANMF	.051 ± .001	.083 ± .001	.007 ± .001	.164 ± .001
M-NMF	.063 ± .001	.084 ± .001	.004 ± .001	.068 ± .001
NNSD	.063 ± .001	.034 ± .001	.004 ± .001	.072 ± .001
SymmNMF	.062 ± .001	.074 ± .001	.007 ± .001	.206 ± .001
Ego-Splitting	.157 ± .001	.202 ± .001	.223 ± .001	.346 ± .001
EdMot	.085 ± .001	.180 ± .001	.008 ± .001	.272 ± .001
LabelProp	.119 ± .001	.090 ± .002	.003 ± .001	.320 ± .004
SCD	.181 ± .001	.189 ± .001	.169 ± .001	.386 ± .001
GEMSEC	.102 ± .001	.127 ± .001	.008 ± .002	.244 ± .001

Tabela 4: Valores médios de AUC com erros padrão em nível de grafo (100 rodadas).

Algoritmo	Reddit Threads	Twitch Egos	GitHub Star	Deezer Egos
GL2Vec	.753 ± .002	.664 ± .002	.551 ± .001	.504 ± .001
Graph2Vec	.804 ± .002	.702 ± .003	.585 ± .001	.512 ± .001
SF	.814 ± .002	.678 ± .003	.558 ± .001	.501 ± .001
NetLSD	.827 ± .001	.631 ± .002	.632 ± .001	.522 ± .001
FGSD	.825 ± .002	.705 ± .003	.656 ± .001	.526 ± .001
GeoScattering	.800 ± .001	.697 ± .001	.546 ± .003	.522 ± .003
FEATHER	.830 ± .002	.720 ± .003	.748 ± .002	.540 ± .001

Tabela 5: Valores médios de AUC com erros padrão em nível de nó (100 rodadas).

Algoritmo	Wikipedia	GitHub	Twitch	FB Page-Page
BoostNE	.685 ± .001	.845 ± .001	.576 ± .001	.752 ± .001
NodeSketch	.722 ± .001	.631 ± .001	.520 ± .001	.579 ± .001
Diff2Vec	.832 ± .001	.858 ± .001	.589 ± .001	.873 ± .001
NetMF	.866 ± .001	.867 ± .001	.629 ± .002	.946 ± .001
Walklets	.875 ± .001	.899 ± .002	.622 ± .001	.973 ± .001
HOPE	.870 ± .001	.844 ± .001	.612 ± .001	.909 ± .001
GraRep	.888 ± .002	.876 ± .001	.609 ± .001	.952 ± .001
DeepWalk	.850 ± .001	.872 ± .002	.597 ± .002	.877 ± .001
NMF-ADMM	.747 ± .001	.784 ± .001	.619 ± .001	.937 ± .001
LAP	.784 ± .001	.529 ± .001	.511 ± .001	.501 ± .001
GraphWave	.517 ± .001	.620 ± .001	.583 ± .001	.613 ± .001
Role2Vec	.845 ± .001	.862 ± .002	.601 ± .002	.903 ± .002
BANE	.866 ± .002	.570 ± .001	.551 ± .001	.970 ± .002
TENE	.907 ± .001	.874 ± .001	.615 ± .001	.886 ± .001
TADW	.896 ± .001	.817 ± .001	.612 ± .002	.871 ± .001
FSCNMF	.912 ± .001	.856 ± .002	.621 ± .001	.891 ± .001
SINE	.904 ± .001	.910 ± .002	.646 ± .001	.979 ± .001
MUSAE	.931 ± .001	.903 ± .001	.628 ± .001	.981 ± .001

4.2 Escalabilidade

Realizamos testes de escalabilidade variando o número de nós, densidade e número de grafos. Sob a ausência de conexão preferencial, todos os métodos incluídos escalam linearmente com o tamanho da entrada, demonstrando a eficiência prática do framework.

5 TRABALHOS RELACIONADOS

Nesta seção, comparamos o Karate Club com bibliotecas de mineração de grafos e aprendizado de máquina.

5.1 Frameworks Orientados a API

A API do Karate Club baseia-se fortemente nas ideias do `scikit-learn` [5], oferecendo configurações padrão sensíveis de hiperparâmetros, encapsulamento através do construtor e suporte a tipos de dados científicos padrão como NumPy e SciPy.

5.2 Bibliotecas de Mineração de Grafos

Ao contrário de bibliotecas como SNAP e GraphTool que requerem pré-requisitos e compilações complexas em C++, o Karate Club possui apenas dependências leves em Python, sendo construído inteiramente sobre o ecossistema do NetworkX. Além disso, as bibliotecas clássicas focam apenas em estatísticas estruturais clássicas ou detecção de comunidades, enquanto o Karate Club cobre de forma abrangente a incorporação de nós e de grafos inteiros.

6 CONCLUSÃO E DIREÇÕES FUTURAS

Neste trabalho, descrevemos o Karate Club, um framework Python que realiza aprendizado não supervisionado em grafos, suportando detecção de comunidades e incorporação de nós e de grafos inteiros. No futuro, planejamos estender o framework para operar em grafos direcionados, ponderados, multiplex e temporais, além de explorar a incorporação hiperbólica de nós.

ACKNOWLEDGMENTS

Benedek Rozemberczki foi apoiado pelo Centre for Doctoral Training in Data Science, financiado pelo EPSRC (bolsa EP/L016427/1).

REFERÊNCIAS

- [1] M. Abadi et al. 2016. Tensorflow: A system for large-scale machine learning. In OSDI '16.
- [2] N. K. Ahmed et al. 2019. role2vec: Role-based network embeddings. In Proc. DLG KDD.
- [3] S. Bandyopadhyay et al. 2018. Fscnmf: Fusing structure and content via non-negative matrix factorization. arXiv:1804.05313.
- [4] M. Belkin e P. Niyogi. 2002. Laplacian eigenmaps and spectral techniques. In NIPS.
- [5] L. Buitinck et al. 2013. API design for machine learning software: experiences from the scikit-learn project. arXiv:1309.0238.
- [6] S. Cao, W. Lu, e Q. Xu. 2015. Grarep: Learning graph representations with global structural information. In CIKM.
- [7] H. Chen e H. Koga. 2019. GL2vec: Graph Embedding Enriched by Line Graphs. In ICONIP.
- [8] N. de Lara e P. Edouard. 2018. A simple baseline algorithm for graph classification. In NeurIPS.
- [9] M. Defferrard et al. PyGSP: Graph Signal Processing in Python. zenodo.1003157.
- [10] C. Donnat et al. 2018. Learning structural node embeddings via diffusion wavelets. In KDD.
- [11] A. Epasto, S. Lattanzi, e R. P. Leme. 2017. Ego-Splitting Framework. In KDD.
- [12] F. Gao, G. Wolf, e M. Hirn. 2019. Geometric Scattering for Graph Data Analysis. In ICMML.
- [13] A. Hagberg, P. Swart, e D. S. Chult. 2008. Exploring network structure, dynamics, and function using NetworkX. Tech Report.
- [14] K. Henderson et al. 2012. Robx: structural role extraction & mining in large graphs. In KDD.
- [15] J. Li, L. Wu, e H. Liu. 2019. Multi-Level Network Embedding. In ASONAM.
- [16] D. Kuang, C. Ding, e H. Park. 2012. Symmetric nonnegative matrix factorization. In SDM.
- [17] J. Leskovec e A. Krevl. 2014. SNAP Datasets: Stanford Large Network Dataset Collection. <http://snap.stanford.edu/data>.
- [18] P.-Z. Li et al. 2019. EdMot: An Edge Enhancement Approach. In KDD.

- [19] A. Narayanan et al. 2017. graph2vec: Learning distributed representations of graphs.
- [20] M. Ou et al. 2016. Asymmetric transitivity preserving graph embedding. In KDD.
- [21] A. Paszke et al. 2019. PyTorch: An imperative style, high-performance deep learning library. In NeurIPS.
- [22] T. P. Peixoto. 2014. The graph-tool python library. figshare.
- [23] B. Perozzi, R. Al-Rfou, e S. Skiena. 2014. Deepwalk: Online learning of social representations. In KDD.
- [24] B. Perozzi et al. 2017. Don't Walk, Skip!: online learning of multi-scale network embeddings. In ASONAM.
- [25] A. Prat-Pérez, D. Dominguez-Sal, e J.-L. Larriba-Pey. 2014. High quality, scalable and parallel community detection. In WWW.
- [26] J. Qiu et al. 2018. Network embedding as matrix factorization. In WSDM.
- [27] U. N. Raghavan, R. Albert, e S. Kumara. 2007. Near Linear Time Algorithm to Detect Community Structures. Phys. Rev. E.
- [28] R. Rehurek e P. Sojka. 2011. Gensim—statistical semantics in python. genism.org.
- [29] B. Rozemberczki, C. Allen, e R. Sarkar. 2019. Multi-scale Attributed Node Embedding. arXiv:1909.13021.
- [30] B. Rozemberczki et al. 2019. GEMSEC: Graph Embedding with Self Clustering. In ASONAM.
- [31] B. Rozemberczki e R. Sarkar. 2018. Fast Sequence-Based Embedding. In Complex Networks.
- [32] B. Rozemberczki e R. Sarkar. 2020. Characteristic Functions on Graphs: Birds of a Feather. In CIKM.
- [33] B.-J. Sun et al. 2017. A non-negative symmetric encoder-decoder approach. In CIKM.
- [34] D. L. Sun e C. Fevotte. 2014. Alternating direction method of multipliers for NMF. In ICASSP.
- [35] A. Tsitsulin et al. 2018. Netlsd: hearing the shape of a graph. In KDD.
- [36] S. Verma e Z.-L. Zhang. 2017. Hunt for the unique, stable, sparse and fast feature learning on graphs. In NeurIPS.
- [37] P. Virtanen et al. 2019. SciPy 1.0—fundamental algorithms for scientific computing in Python. arXiv:1907.10121.
- [38] S. van der Walt, S. C. Colbert, e G. Varoquaux. 2011. The NumPy array. CSE.
- [39] X. Wang et al. 2017. Community Preserving Network Embedding. In AAAI.
- [40] C. Yang et al. 2015. Network representation learning with rich text information. In IJCAI.
- [41] C. Yang et al. 2017. Fast network embedding enhancement. In IJCAI.
- [42] D. Yang et al. 2019. NodeSketch: Highly-Efficient Graph Embeddings. In KDD.
- [43] H. Yang et al. 2018. Binarized attributed network embedding. In ICDM.
- [44] F. Ye, C. Chen, e Z. Zheng. 2018. Deep Autoencoder-like Nonnegative Matrix Factorization. In CIKM.
- [45] W. W. Zachary. 1977. An information flow model for conflict and fission in small groups. JAR.
- [46] D. Zhang et al. 2018. SINE: Scalable Incomplete Network Embedding. In ICDM.